Machine Coding / System Design Assignment: React Native

Position: SDE III — Frontend / React Native
Time: 3 Hours
Primary Focus: Frontend Architecture, Platform Design, Extensibility & Reliability

Overview

MyNaksh is evolving from an astrology application into a broader **AI-powered personal intelligence platform**.

Today, users may interact with:

- AI Astrology
- Tarot
- Gemstone Recommendations
- Remedies
- Panchang
- Human Astrologer Consultations
- Personalized Reports
- Marketplace Recommendations

In the future, MyNaksh may introduce many more AI-powered experiences and agents.

Different product and engineering teams will own different experiences. We therefore want a frontend architecture that allows new experiences to be added and evolved **without repeatedly changing the core application**.

Your task is to build a simplified **AI Experience Orchestration Layer** in React Native.

The focus is not on visual polish. We want to understand how you think about **platform architecture, boundaries, extensibility and production reliability**.

Part A — Build the Experience Runtime

Build a small **Experience Runtime** that receives an experience definition from a mock backend and renders the appropriate experience.

For example:

```
{
  "experience": "tarot",
  "version": "v1",
  "entryPoint": "chat",
  "config": {
    "deck": "classic",
    "maxCards": 3
  }
}
```

Another experience could be:

```
{
  "experience": "gemstone",
  "version": "v1",
  "entryPoint": "chat",
  "config": {
    "recommendationMode": "personalized"
  }
}
```

Your implementation must demonstrate:

1. Experience Registry

Create a mechanism through which experiences register themselves with the runtime.

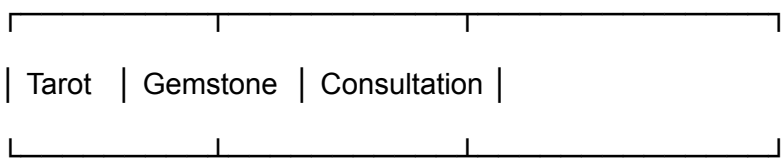
For example:

Experience Runtime

↓

Experience Registry

↓



Avoid putting all experience-specific business logic into one central component.

2. Experience Routing

The runtime should receive an experience definition and route it to the correct implementation.

Implement at least **3 experiences**:

- Tarot
- Gemstone Recommendation
- Astrologer Consultation

The UI can be intentionally simple.

Architecture is more important than visual design.

3. Configuration

Experience-specific configuration should be passed to the experience rather than interpreted by the core runtime.

For example:

```
{
  "experience": "tarot",
  "config": {
    "deck": "classic",
    "maxCards": 3
  }
}
```

The Tarot experience should own the meaning of `deck` and `maxCards`.

The runtime should not contain Tarot-specific business logic.

4. Versioning

Demonstrate how your architecture supports experience versions such as:

tarot/v1

tarot/v2

You do **not** need to build two complete versions.

The implementation should make it clear where version resolution belongs and how different versions could coexist.

5. Common Lifecycle & Failure Handling

Define a reasonable common lifecycle, for example:

Initializing

↓

Loading

↓

Ready

↓

Completed / Failed

An experience should be able to fail independently.

For example:

AI Experiences

├── Tarot ✅

├── Gemstone ❌ API failure

└── Consultation ✅

Use mock APIs to demonstrate at least one experience-level failure without breaking the rest of the application.

Mock Backend Data

You may use the following as a starting point:


```
{
  "experiences": [
    {
      "experience": "tarot",
      "version": "v1",
      "entryPoint": "chat",
      "enabled": true,
      "config": {
        "deck": "classic",
        "maxCards": 3
      }
    },
    {
      "experience": "gemstone",
      "version": "v1",
      "entryPoint": "chat",
      "enabled": true,
      "config": {
        "recommendationMode": "personalized"
      }
    },
    {
      "experience": "consultation",
      "version": "v1",
      "entryPoint": "chat",
      "enabled": true,
      "config": {
        "mode": "human_astrologer"
      }
    }
  ]
}
```


You are free to modify or extend this structure if required by your architecture.

Expected Architecture

A possible high-level flow is:

Backend Configuration

↓

Experience Runtime

↓

Experience Registry

↓

Version Resolution

↓

Experience Implementation

↓

Experience-specific State / API

↓

UI

This is **not a prescribed implementation**.

We want to see how you decide where responsibilities belong.

Constraints

- **Time:** 3 hours
- Partial completion is completely acceptable.
- Prioritize **architecture and engineering decisions** over visual polish.
- Backend implementation is not required.
- Use mock APIs/data.
- **TypeScript is preferred.**
- React Native should be used.
- Avoid unnecessary third-party libraries.
- You do not need to build production infrastructure.
- Do not spend significant time on animations or visual design.

Submission Requirements

1. Downloadable zip (preferred) or GitHub Repository

Please submit repository containing:

- Source code
- Mock API/data layer
- README

2. Short Screen Recording

Record a **2–5 minute walkthrough** showing:

- Experience Runtime
- Three experiences
- Experience registration
- Configuration flow
- Versioning approach
- Failure handling

3. README

Please explain:

- Overall architecture
- Experience registration
- Module/component boundaries
- State and data flow
- Versioning strategy
- Failure handling
- How a new team would add an experience
- Major trade-offs and decisions